

3. Making Prediction: Decoding and Inference

3.1. Linear models:

3.1.1. Discriminative models: $\Rightarrow$ CRFs

- Decoding: **Viterbi**,
- Inference: **Forward**

3.1.2. Generative models: $\Rightarrow$ HMMs

- Decoding: **Viterbi**,
- Inference: **Forward**

3.2. Non-linear (generative) models: $\Rightarrow$ PCFGs

- Decoding: probability of most likely parse tree: **Viterbi**
- Inference: probability of a string: **Inside**
- Weaknesses of PCFGs as parsing models

Given $x = x_1, x_2, \dots, x_T \in \mathcal{X}^*$ and $y = y_1, y_2, \dots, y_T \in \mathcal{Y}^*$



Discriminative models: **Conditional Random Fields (CRF)**

[Lafferty, McCallum, Pereira, 2001].

$$p(y|x; \theta) = \frac{\exp\left\{\sum_{t=1}^T \sum_{k=1}^K \theta_k f_k(y_{t-1}, y_t, x_t)\right\}}{Z(x; \theta)} \quad (4)$$

Where

$$Z(x; \theta) = \sum_{y' \in \mathcal{Y}^*} \exp\left\{\sum_{t=1}^T \sum_{k=1}^K \theta_k f_k(y'_{t-1}, y'_t, x_t)\right\} \quad (5)$$

CRFs AS FACTOR GRAPHS

We can define a transition (factor) as:

$$\Psi_t(y_{t-1}, y_t, x_t) \stackrel{\text{def}}{=} \exp\left\{\sum_{k=1}^K \theta_k f_k(y_{t-1}, y_t, x_t)\right\}$$

From (4) CRFs can be defined as:

$$p(y|x; \theta) = \frac{1}{Z(x; \theta)} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

Also from (5)

$$Z(x; \theta) = \sum_{y' \in \mathcal{Y}^*} \prod_{t=1}^T \Psi_t(y'_{t-1}, y'_t, x_t)$$

CRFs AS FACTOR GRAPHS

- **Decoding:** Given θ and x , predict the best output sequence for x .

$$\hat{y} = \arg \max_y p(y|x; \theta) = \arg \max_y \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

- **The probability of an output sequence**

$$p(y|x; \theta) = \frac{1}{Z(x; \theta)} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

DECODING WITH CRFs: VITERBI ALGORITHM

➤ **Definition:** Score of optimal sequence for $x_1 \dots x_t$ ending at $y_t = s \in \mathcal{Y}$

$$\gamma_t(s) \stackrel{\text{def}}{=} \max_{y_1^t; y_t=s} \prod_{i=1}^t \Psi_i(y_{i-1}, y_i, x_i)$$

➤ **Initialization:** $\forall s \in \mathcal{Y}$

$$\gamma_1(s) = \Psi_1(y_0 = \text{null}, y_1 = s, x_1)$$

➤ **Recursion:** $\forall t = 2 \dots T$; and $\forall s \in \mathcal{Y}$

$$\gamma_t(s) = \max_{s' \in \mathcal{Y}} \left\{ \gamma_{t-1}(s') \cdot \Psi_t(y_{t-1} = s', y_t = s, x_t) \right\}$$

➤ **Final result:** The optimal score for x is:

$$\max_s \gamma_T(s)$$

Backpointers can be used to recover the optimal sequence $\hat{y}$

INFERENCE WITH CRFs: FORWARD ALGORITHM

➤ **Definition:** Score for $x_1 \dots x_t$ ending at $y_t = s \in \mathcal{Y}$

$$\alpha_t(s) \stackrel{\text{def}}{=} \sum_{y_1^t; y_t=s} \prod_{i=1}^t \Psi_i(y_{i-1}, y_i, x_i)$$

➤ **Initialization:** $\forall s \in \mathcal{Y}$

$$\alpha_1(s) = \Psi_1(y_0 = \text{null}, y_1 = s, x_1)$$

➤ **Recursion:** $\forall t = 2 \dots T$; and $\forall s \in \mathcal{Y}$

$$\alpha_t(s) = \sum_{s' \in \mathcal{Y}} \alpha_{t-1}(s') \cdot \Psi_t(y_{t-1} = s', y_t = s, x_t)$$

➤ **Final result:** The optimal score for x is:

$$\sum_s \alpha_T(s)$$

CRFs AS FACTOR GRAPHS

➤ The probability of an output sequence

$$p(y|x; \theta) = \frac{1}{Z(x; \theta)} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

Where we can compute $Z(x; \theta)$ efficiently using the forward algorithm

$$Z(x; \theta) = \sum_{y_1^T} \prod_{t=1}^T \Psi_t(y'_{t-1}, y'_t, x_t) = \sum_s \alpha_T(s)$$

HIDDEN MARKOV MODELS

Given $x = x_1, x_2, \dots, x_T \in \mathcal{X}^*$ and $y = y_1, y_2, \dots, y_T \in \mathcal{Y}^*$

Generative models: Hidden Markov Models $(\mathcal{Q}, \Sigma, \theta)$, where $\theta = (q, e)$

$$P_\theta(x_1^T, y_1^T) = \prod_{t=1}^T q(y_t|y_{t-1}) \cdot e(x_t|y_t)$$

➤ Probability of the observation

$$P_\theta(x_1^T) = \sum_{y_1^T} P_\theta(x_1^T, y_1^T)$$

➤ Probability of the best state sequence

$$\hat{P}_\theta(x_1^T) = \max_{y_1^T} P_\theta(x_1^T, y_1^T)$$

➤ Finding the best state sequence

$$\hat{y}_1^T = \arg \max_{y_1^T} P_\theta(x_1^T, y_1^T)$$

HMMS AS FACTOR GRAPHS

$$\Psi_t(y_{t-1}, y_t, x_t) = q(y_t|y_{t-1}) \cdot e(x_t|y_t)$$

HMMs can be defined in terms of transitions (factors) $\Psi_t(y_{t-1}, y_t, x_t)$, such as:

$$P_\theta(x_1^T, y_1^T) \stackrel{\text{def}}{=} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

➤ **Probability of the observation**

$$P_\theta(x_1^T) = \sum_{y_1^T} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

➤ **Probability of best the state sequence**

$$\hat{P}_\theta(x_1^T) = \max_{y_1^T} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

➤ **Finding the best state sequence**

$$\hat{y}_1^T = \arg \max_{y_1^T} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

VITERBI ALGORITHM

➤ **Definition:** The maximum probability that state $y_t = s \in \mathcal{Y}$ is reached at time t by emitting the prefix x_1^t .

$$\gamma_t(s) \stackrel{\text{def}}{=} \max_{y_1^t; y_t=s} \prod_{i=1}^t \Psi_i(y_{i-1}, y_i, x_i) \equiv \max_{y_1^t; y_t=s} \hat{P}_\theta(x_1^t, y_1^t)$$

➤ **Initialization:** $\forall s \in \mathcal{Y}$

$$\gamma_1(s) = \Psi_1(\text{null}, s, x_1)$$

➤ **Recursion:** $\forall i = 2 \dots T$; and $\forall s \in \mathcal{Y}$

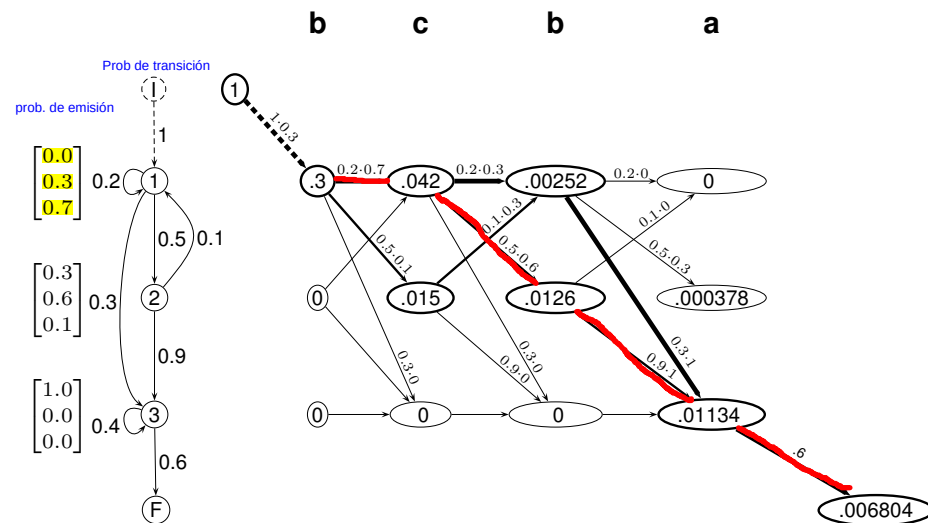
$$\gamma_t(s) = \max_{s' \in \mathcal{Y}} \{ \gamma_{t-1}(s') \cdot \Psi_t(s', s, x_t) \}$$

➤ **Final result:** The probability of the best state sequence for x_1^T given the model is:

$$\hat{P}_\theta(x_1^T) = \max_s \gamma_T(s)$$

Backpointers can be used to recover the optimal sequence $\hat{y}_1^T$

ALGORITMO DE VITERBI: EJEMPLO



FORWARD ALGORITHM

➤ **Definition:** The probability of generating the prefix x_1^t , reaching the state $y_t = s \in \mathcal{Y}$ at time t .

$$\alpha_t(s) \stackrel{\text{def}}{=} \sum_{y_1^t; y_t=s} \prod_{i=1}^t \Psi_i(y_{i-1}, y_i, x_i) \equiv \sum_{y_1^t; y_t=s} P_\theta(x_1^t, y_1^t)$$

➤ **Initialization:** $\forall s \in \mathcal{Y}$

$$\alpha_1(s) = \Psi_1(\text{null}, s, x_1)$$

➤ **Recursion:** $\forall i = 2 \dots T$; and $\forall s \in \mathcal{Y}$

$$\alpha_t(s) = \sum_{s' \in \mathcal{Y}} \alpha_{t-1}(s') \cdot \Psi_t(s', s, x_t)$$

➤ **Final result:** The probability of the observation x_1^T given the model is:

$$P_\theta(x_1^T) = \sum_s \alpha_T(s)$$

$$\Psi_t(y_{t-1}, y_t, x_t) = q(y_t|y_{t-1}) \cdot e(x_t|y_t)$$

HMMs can be defined in terms of transitions (factors) $\Psi_t(y_{t-1}, y_t, x_t)$, such as:

$$P_\theta(x_1^T, y_1^T) = \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t)$$

➤ Probability of the observation

$$P_\theta(x_1^T) = \sum_{y_1^T} P_\theta(x_1^T, y_1^T) = \sum_{y_1^T} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t) = \sum_s \alpha_T(s)$$

➤ Probability of the best state sequence

$$\hat{P}_\theta(x_1^T) = \max_{y_1^T} P_\theta(x_1^T, y_1^T) = \max_{y_1^T} \prod_{t=1}^T \Psi_t(y_{t-1}, y_t, x_t) = \max_s \gamma_T(s)$$

Generative models: Probabilistic Context-Free Grammars

$$P_\theta(x, t_x) = \prod_{i=1}^m p(r_i) \quad \text{where } t_x : r_1, \dots, r_m$$

➤ Probability of an observation

$$P_\theta(x) = \sum_{t_x \in \mathcal{T}_x} P_\theta(x, t_x)$$

➤ Probability of the best parse tree

$$\hat{P}_\theta(x) = \max_{t_x \in \mathcal{T}_x} P_\theta(x, t_x)$$

➤ Finding the best parse

$$\hat{t}_x = \arg \max_{t_x \in \mathcal{T}_x} P_\theta(x, t_x)$$

Viterbi ALGORITHM

➤ **Definition:** Given $x = x_1 \dots x_T \in \Sigma^*$ and $A \in N$

$$\hat{e}(A, i, i+l) \stackrel{\text{def}}{=} \hat{P}_\theta(A \xrightarrow{*} x_{i+1} \dots x_{i+l})$$

➤ **Initialization:** $\forall A \in N; \quad \forall i : 0 \dots T-1;$

$$\hat{e}(A, i, i+1) = p(A \rightarrow b) \cdot \delta(b, x_{i+1})$$

➤ **Recursion:** $\forall A \in N; \quad \forall l : 2 \dots T; \quad \forall i : 0 \dots T-l;$

$$\hat{e}(A, i, i+l) = \max_{B, C \in N} \left\{ p(A \rightarrow BC) \cdot \max_{k=1, \dots, l-1} \{ \hat{e}(B, i, i+k) \cdot \hat{e}(C, i+k, i+l) \} \right\}$$

➤ **Final result:** The probability of the best parse tree is:

$$\hat{P}_\theta(x) = \hat{e}(S, 0, T)$$

Backpointers can be used to recover the optimal sequence $\hat{y}$

Inside ALGORITHM

➤ **Definition:** Given $x = x_1 \dots x_T \in \Sigma^*$ and $A \in N$

$$e(A, i, i+l) \stackrel{\text{def}}{=} P_\theta(A \xrightarrow{*} x_{i+1} \dots x_{i+l})$$

➤ **Initialization:** $\forall A \in N; \quad \forall i : 0 \dots T-1;$

$$e(A, i, i+1) = p(A \rightarrow b) \cdot \delta(b, x_{i+1})$$

➤ **Recursion:** $\forall A \in N; \quad \forall l : 2 \dots T; \quad \forall i : 0 \dots T-l;$

$$e(A, i, i+l) = \sum_{B, C \in N} \left\{ p(A \rightarrow BC) \cdot \sum_{k=1, \dots, l-1} e(B, i, i+k) \cdot e(C, i+k, i+l) \right\}$$

➤ **Final result:** The sentence probability is:

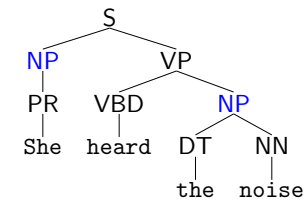
$$P_\theta(x) = e(S, 0, T)$$

WEAKNESSES OF PCFGs AS PARSING MODELS

➤ Poor independence assumptions:

⇒ Parsing with PCFG considers conditional independence in the application of the rules

WEAKNESSES OF PCFGs AS PARSING MODELS



- Independence assumptions are often too strong
- Example: the expansion of an NP is highly dependent on the parent (i.e., subjects vs. objects) Statistics for NPs in the Switchboard corpus [Francis et al., 1999]

	Pronoun	Non-pronoun
Subject	91 %	9 %
Object	34 %	66 %

Also, the subject and object expansions are correlated !

One way to tackle it: splitting the non-terminals

WEAKNESSES OF PCFGs AS PARSING MODELS

➤ Poor independence assumptions:

⇒ Parsing with PCFG considers conditional independence in the application of the rules

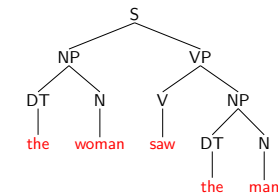
➤ Lack of lexical conditioning:

⇒ There are lexical dependencies in the syntax analysis (the probability of a syntactic structure can vary depending on the words involved)

WEAKNESSES OF PCFGs AS PARSING MODELS

➤ Probabilistic CFGs:

$$\begin{aligned}
 S &\rightarrow NP \ VP \\
 VP &\rightarrow V \ NP \\
 NP &\rightarrow DT \ N
 \end{aligned}$$



The expansion of a verb phrase VP as a verb followed by one or two noun phrases is independent of the choice of verb involved.

➤ Example: [Manning and Schütze, 1999]

Local rules	verb			
	come	take	think	want
VP → V	9.5 %	2.6 %	4.6 %	5.7 %
VP → V NP	1.1 %	32.1 %	0.2 %	13.9 %
VP → V PP	34.5 %	3.1 %	7.1 %	0.3 %
VP → V SBAR	6.6 %	0.3 %	73.0 %	0.2 %
VP → V S	2.2 %	1.3 %	4.8 %	70.8 %
VP → V NP S	0.1 %	5.7 %	0.0 %	0.3 %
VP → V PRT NP	0.3 %	5.8 %	0.0 %	0.0 %
VP → V PRT PP	6.1 %	1.5 %	0.2 %	0.0 %

One way to tackle it: lexicalized models

➤ Poor independence assumptions:

⇒ Parsing with PCFG considers conditional independence in the application of the rules

➤ Lack of lexical conditioning:

⇒ There are lexical dependencies in the syntax analysis (the probability of a syntactic structure can vary depending on the words involved)

➤ It is not considered the context

⇒ The context (eg, slang, knowledge of the subject, etc.) is necessary for proper disambiguation of language